

LogiEdge: Intelligent Edge AI Platform for Cold-Chain Logistics

Authors:

Name	Id	Contribution
RAJESH A S	2024ac05683@wilp.bits-pilani.ac.in	100%
Pavan Kumar S	2024ac05682@wilp.bits-pilani.ac.in	100%
DineshKumar P	2024ac05562@wilp.bits-pilani.ac.in	100%
Sachin Deepak K	2024ac05608@wilp.bits-pilani.ac.in	100%

Links:

Source code Github: <https://github.com/PavanKumar-WILP/logibridge.git>

Demo Video:

https://drive.google.com/drive/folders/19H4s_0iOVaI6ZdRAbyKI5efIC0zoU3Wt?usp=sharing

Table of Contents

Phase 1 Report.....	3
1 System Architecture and Deployment	3
1.1 Task A1 – Constraint Analysis	3
1.2 Task A2 – System Architecture Diagram	6
2 Hardware Selection & Justification	8
2.1 Task B1 – Hardware Selection using the Constraint Triangle	8
2.2 Task B2 – Arithmetic Intensity and Roofline Analysis	13

LogiEdge: Intelligent Edge AI Platform for Cold-Chain Logistics

Phase 1 Report

1 System Architecture and Deployment

1.1 Task A1 – Constraint Analysis

FreightBridge Logistics operates 85 refrigerated trucks transporting temperature-sensitive pharmaceutical products where maintaining an uninterrupted cold chain is essential. The proposed LogiEdge system must satisfy the four fundamental Edge AI constraints—latency, bandwidth, connectivity, and privacy—to ensure reliable operation under real-world Indian transportation conditions.

1. Latency Constraint

The refrigeration unit may fail unexpectedly, causing the cargo temperature to increase at approximately **1°C per minute**. The project specifies that the system must detect the fault and generate an alert within **90 seconds** of the fault signature appearing in the sensor data.

A cloud-only architecture cannot consistently satisfy this Service Level Agreement (SLA). In rural regions of Maharashtra and Andhra Pradesh, cellular latency typically ranges between **150–400 ms** under good 4G coverage and can exceed **1–3 seconds** during weak coverage or network congestion. Each inference would require:

- Sensor transmission from truck to cloud
- Cloud processing
- Alert transmission back to the operations centre
- Network retries during unstable connectivity

Although the network latency alone may appear acceptable under ideal conditions, continuous cloud dependence introduces unpredictable delays and complete service interruption during connectivity loss. Edge inference eliminates these uncertainties by executing the machine learning model locally on the truck. Since inference occurs directly on the edge device, only the final alert is transmitted, enabling deterministic response well within the 90-second requirement.

2. Bandwidth Constraint

Each refrigerated truck continuously generates three sensor streams.

Temperature

- Sampling frequency = 1 Hz
- 86,400 readings/day
- Assuming 4-byte float:
 $86,400 \times 4 = \mathbf{345,600 \text{ bytes}}$

Vibration

- 500 Hz
- 3-axis accelerometer
- 3 values/sample
- 4 bytes/value

Daily samples:

$$500 \times 3 \times 86,400 = \mathbf{129,600,000 \text{ values}}$$

Daily data:

$$129,600,000 \times 4 = \mathbf{518,400,000 \text{ bytes} \approx 518.4 \text{ MB}}$$

Door Events

Assuming approximately 100 events/day:

$$100 \times 32 \text{ bytes} \approx \mathbf{3.2 \text{ KB}}$$

Total Raw Data

- Temperature = **0.35 MB**
- Vibration = **518.40 MB**
- Door Events = **0.003 MB**

Total ≈ 518.75 MB per truck per day

Transmission cost:

$518.75 \times ₹0.10 = ₹51.88/\text{day per truck}$

For 85 trucks:

- $₹51.88 \times 85 = ₹4,409.80/\text{day} = 4,410 \text{ per day and } 1,32,300 / \text{month}$

By contrast, edge AI transmits only anomaly alerts and periodic summaries. Assuming **100 alerts/day** with an average payload of **1 KB**, daily transmission becomes only **100 KB (0.1 MB)**.

Cost:

$0.1 \times ₹0.10 = ₹0.01/\text{day per truck}$

Thus, edge processing reduces communication bandwidth by more than **99.98%**, significantly lowering operational costs.

3. Connectivity Constraint

The Nashik–Aurangabad route experiences documented cellular outages lasting **35–90 minutes** at seven locations. During these outages, a cloud-only system cannot receive sensor data, perform inference, or generate alerts. Consequently, refrigeration failures may remain undetected until connectivity is restored, potentially causing complete vaccine spoilage.

The proposed LogiEdge architecture performs sensor acquisition, preprocessing, feature extraction, and inference locally. Alerts are stored within a local persistent alert database while offline. Once cellular connectivity resumes, all buffered alerts are synchronised automatically with the operations centre. Therefore, safety monitoring continues uninterrupted regardless of network availability.

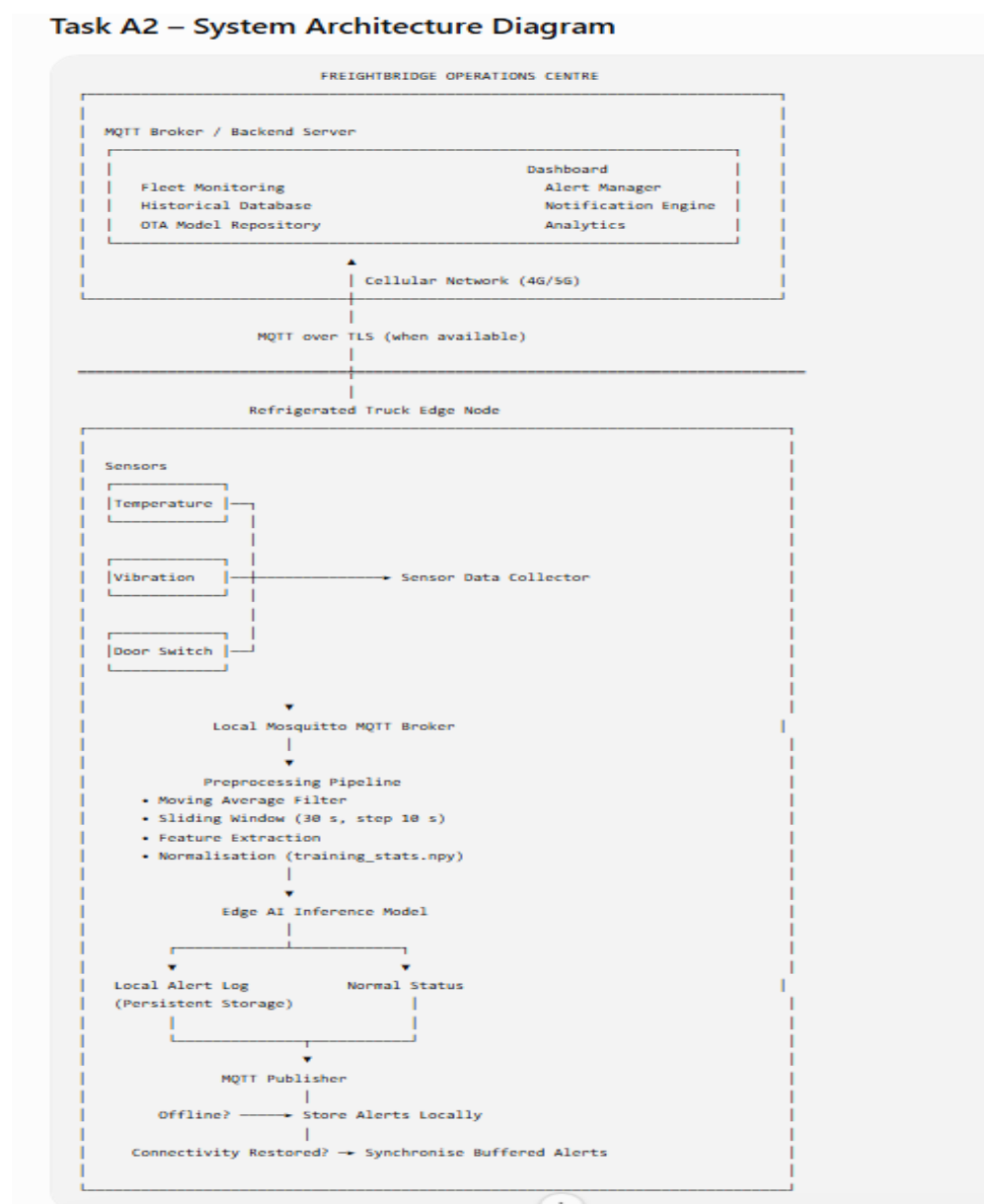
LogiEdge decouples inference execution from network connectivity. On-device TFLite models process sensor windows continuously, persisting classifications and alerts to an append-only local SQLite/WAL log during network outages. Once cellular coverage returns, the edge node automatically synchronizes buffered logs with the cloud backend via MQTT with QoS 1 guarantees.

4. Privacy Constraint

FreightBridge's pharmaceutical customers require assurance that temperature history and cold-chain records remain confidential. A cloud-centric architecture continuously uploads raw sensor streams over public cellular networks, increasing exposure to interception, unauthorized access, and regulatory compliance risks.

Edge AI performs inference directly inside the truck, meaning raw temperature, vibration, and operational sensor data never leave the vehicle during normal operation. Only compact inference results, anomaly classifications, timestamps, and alert metadata are transmitted using encrypted MQTT communication. This significantly reduces the attack surface, protects sensitive pharmaceutical logistics information, and satisfies contractual requirements for secure cold-chain monitoring.

1.2 Task A2 – System Architecture Diagram



Architecture Explanation

1. Temperature, vibration, and door sensors continuously generate real-time measurements.
2. All sensor data are first published to a **local Mosquitto MQTT broker** running on the truck.
3. The preprocessing pipeline performs filtering, feature extraction, and normalization.
4. The trained Edge AI model classifies the truck state as **Normal, Warning, or Critical**.
5. Critical events are immediately stored in the **local alert log**, ensuring no alerts are lost during communication failures.
6. When cellular connectivity is available, the MQTT publisher securely synchronizes alerts to the FreightBridge Operations Centre.
7. During network outages, inference continues entirely on the truck, satisfying the project's requirement for **fully offline operation** with delayed synchronization after connectivity returns.

2 Hardware Selection & Justification

2.1 Task B1 – Hardware Selection using the Constraint Triangle

Hardware Options

Three hardware configurations were evaluated against the pilot requirements:

Option	Edge Hardware Platform	Unit Cost (INR)	TDP / Power	Compute Capability
1	Raspberry Pi 5 (8GB) + AI HAT+	~₹15,000	7.5W	13 TOPS (Hailo-8L) + 16 GFLOP/s CPU
2	Jetson Orin Nano Super Dev Kit	~₹45,000	15W	67 TOPS
3	STM32H7-based Custom MCU	~₹3,500	0.4W	No AI accelerator Ultra-low power / Limited SRAM

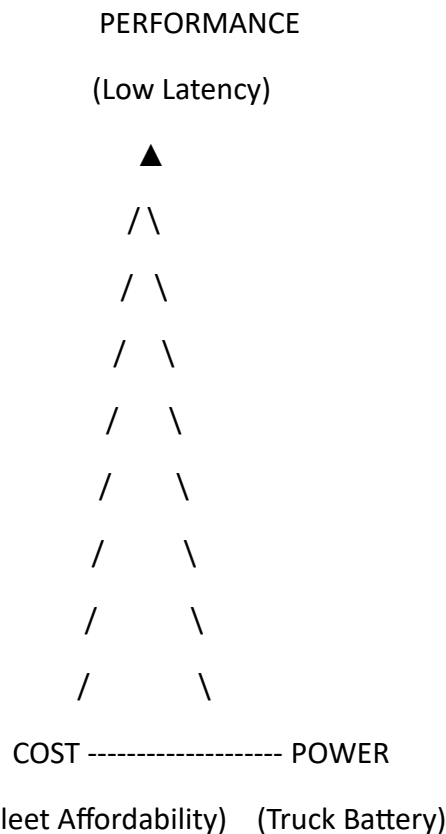
The Edge AI hardware must satisfy three competing constraints:

1. **Performance (Latency)**
2. **Power Consumption**

3. Cost

These form the **Constraint Triangle**. Improving one constraint generally impacts the other two.

Constraint Triangle Analysis



For FreightBridge, **Performance** is the dominant constraint because failure to detect refrigeration problems within 90 seconds may result in pharmaceutical spoilage worth several lakhs of rupees. However, the selected hardware must also remain below the specified **10 W AI power budget** and be economically feasible for deployment across **85 pilot trucks**, eventually expanding to **265 trucks**.

Option 1 – Raspberry Pi 5 + Hailo-8L AI HAT+

Advantages

The Raspberry Pi 5 equipped with the Hailo-8L AI accelerator provides approximately **13 TOPS**, which is significantly higher than the computational requirement of the proposed MLP classifier. The inference workload is only **45 MFLOPs**, meaning inference completes in only a few milliseconds, comfortably satisfying the 90-second detection SLA.

Power consumption is approximately **7.5 W**, remaining safely below the project's **10 W power budget**. Since refrigerated trucks operate using a **12 V vehicle battery connected through a DC–DC converter**, this power requirement has negligible impact on vehicle operation.

The total deployment cost is reasonable.

Fleet Cost

Pilot deployment (85 trucks)

₹15,000 × 85

= ₹12,75,000

Full deployment (265 trucks)

₹15,000 × 265

= ₹39,75,000

This investment is justified considering that a single vaccine spoilage incident already resulted in losses of **₹28 lakh**. Preventing even two such incidents nearly offsets the hardware investment.

Disadvantages

The Raspberry Pi platform consumes more power than an MCU-based solution and requires Linux maintenance. However, these disadvantages are outweighed by its AI capability and deployment flexibility.

Option 2 – NVIDIA Jetson Orin Nano Super

Advantages

The Jetson Orin Nano delivers approximately **67 TOPS**, making it capable of running very large deep learning models including CNNs, Transformers, and object detection networks.

Inference latency would be extremely low.

Large RAM and GPU acceleration also provide future scalability.

Disadvantages

The hardware exceeds the project's power constraint.

Power consumption:

15 W

Project AI power budget:

10 W

Therefore,

$$15\text{ W} > 10\text{ W}$$

making it unsuitable for installation inside the truck without redesigning the electrical system.

Cost is another major issue.

Pilot fleet:

$$₹45,000 \times 85$$

$$= ₹38,25,000$$

Full deployment:

$$₹45,000 \times 265$$

$$= ₹1,19,25,000$$

This is nearly **three times** the cost of the Raspberry Pi solution while providing computational capability that is unnecessary for a lightweight MLP model.

Therefore, Jetson provides excessive performance that the application never utilizes.

Option 3 – STM32H7 Custom MCU**Advantages**

The STM32H7 has the lowest cost and power consumption.

Power:

$$0.4\text{ W}$$

Pilot deployment:

$$₹3,500 \times 85$$

$$= ₹2,97,500$$

Full deployment:

$$₹3,500 \times 265$$

$$= ₹9,27,500$$

It is an excellent choice for simple embedded control systems.

Disadvantages

The STM32H7 lacks a dedicated AI accelerator.

Executing feature extraction, MQTT communication, local storage, Docker-style software architecture, and machine learning inference simultaneously is beyond the practical capability of the MCU.

Model updates, OTA deployment, PSI monitoring, Docker containers, and Linux-based MLOps are also difficult to implement on a microcontroller platform.

Although the power and cost are attractive, the platform cannot reliably satisfy the project's functional requirements.

Dominant Constraint Vertex

For the FreightBridge deployment, the dominant vertex of the Constraint Triangle is **Performance (Latency)**.

The system must detect refrigeration faults within **90 seconds**, continue operating during network outages, and execute AI inference locally. Therefore, computational capability is the primary design consideration. However, this performance must be achieved while respecting the **10 W power limit** and maintaining a fleet-scale deployment cost suitable for 85 pilot trucks and future expansion to 265 vehicles.

Final Hardware Recommendation

The **Raspberry Pi 5 with Hailo-8L AI HAT+** is the most appropriate hardware platform because it provides the best balance between performance, power efficiency, and deployment cost.

It satisfies all project constraints:

- Meets the **90-second latency requirement** with millisecond-level inference.
- Operates within the **10 W AI power budget** (7.5 W).
- Supports Linux, Docker, MQTT, TensorFlow Lite, and Ansible required by the project.
- Enables OTA updates, MLOps monitoring, and offline edge inference.
- Provides a scalable and economically viable solution for both the **85-truck pilot** and the future **265-truck fleet**.

The Jetson Orin Nano offers unnecessary computational power at significantly higher cost and power consumption, while the STM32H7 lacks sufficient AI capability for the required Edge AI workload.

Note: In this project we the group members are located in different geo locations (like Bangalore, Coimbatore), hence a common development platform (like Raspberry Pi) cannot be used. So, we are using our laptop machines(X64) as the edge development platform which allows parallel code development and validation by individual team members and avoids the dependency on hardware availability. Regular sync meetings are used for architectural design sync, code development and validation related discussions.

2.2 Task B2 – Arithmetic Intensity and Roofline Analysis

The trained machine learning model performs:

- Computational workload = **45 MFLOPs**
- Memory accessed = **18 MB**
- Raspberry Pi 5 CPU Performance = **16 GFLOP/s**
- Memory Bandwidth = **12 GB/s**

Step 1 – Calculate Arithmetic Intensity

Arithmetic Intensity (AI) is defined as:

$$I = \text{Total Floating-point operations} / \text{Total Memory Access}$$

$$45 \times 10^6 \text{ FLOPs} / 18 \times 10^6 \text{ Bytes} = 2.50 \text{ FLOP/byte}$$

Arithmetic Intensity = 2.5 FLOPs/Byte

Step 2 – Calculate Roofline Ridge Point

The ridge point separates memory-bound and compute-bound execution.

CPU Ridge Point (I_{ridge}) Calculation

$$I_{\text{ridge}} = \text{Peak Compute Performance} / \text{Peak Memory Bandwidth}$$

$$= 16 \times 10^9 \text{ FLOP/s} / 12 \times 10^9 \text{ Bytes/s} = 1.33 \text{ FLOP/byte}$$

Ridge Point = 1.33 FLOPs/Byte

Step 3 – Compare Arithmetic Intensity with Ridge Point

Calculated Arithmetic Intensity

2.5 FLOPs/Byte

Ridge Point

1.33 FLOPs/Byte

Since $2.5 > 1.33$

the model lies **to the right of the ridge point** on the Roofline graph.

Therefore, the workload is **compute-bound** rather than memory-bandwidth bound.

This means that the processor spends more time performing mathematical computations than waiting for data to be fetched from memory.

Step 4 – Roofline Classification

Metric	Value
Floating-point operations	45 MFLOPs
Memory accessed	18 MB
Arithmetic Intensity	2.5 FLOPs/Byte
Peak Compute Performance	16 GFLOP/s
Memory Bandwidth	12 GB/s
Ridge Point	1.33 FLOPs/Byte
Classification	Compute-bound

Step 5 – Latency Optimisation Based on the Roofline Model

Since the workload is compute-bound, increasing memory bandwidth would have minimal effect on inference latency. Instead, latency improvements should focus on reducing the amount of computation performed.

Suitable optimisation techniques include:

- **Post-Training INT8 Quantisation**, which replaces FP32 operations with INT8 arithmetic to reduce computation time.

- **Structured pruning**, which removes redundant neurons or filters, reducing the number of floating-point operations while preserving model accuracy.
- **Leveraging the Hailo-8L AI accelerator** instead of executing inference solely on the Raspberry Pi CPU, significantly increasing throughput and lowering latency.
- **Model simplification**, such as reducing hidden-layer size or eliminating unnecessary parameters, provided the validation accuracy remains above the required **88%** and the **Critical (Class 2) recall exceeds 95%**.

These optimizations move the execution closer to the hardware's peak compute capability, reducing inference latency without being constrained by memory bandwidth.